

Svelte **Review Cards**



Eight concepts an interviewer can probe and a beginner must own.
Read the question, cover the answer, answer aloud, then check the
code and the figure.

1 — Reactivity: \$state

STATE MANAGEMENT

Q. How do you declare a variable that automatically updates the DOM when its value changes in Svelte 5?

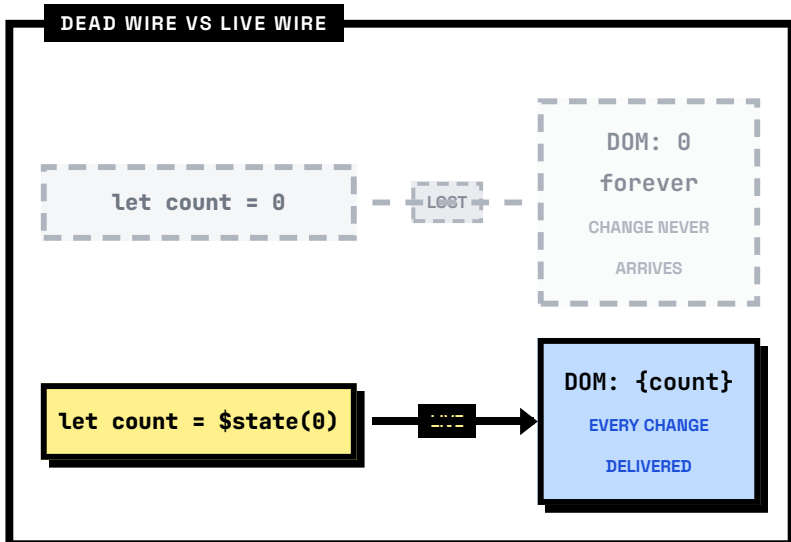
ANSWER. Normally, JavaScript variables are completely dead: changing a `let` variable does not update your UI. You use the `$state` rune to wrap your initial value, turning it into a live variable that Svelte tracks. Any time that value changes, Svelte instantly updates the DOM for you.

App.svelte

```
<script>
  let count = $state(0);
  let user = $state({ name: 'Nikos' });

  function update() {
    count++;
    user.name = 'Hugo'; // Deeply reactive
  }
</script>
```

DIAGRAM CONCEPT. Dead Wire vs Live Wire: a plain `let` is a dead wire, change it and nothing reaches the page; `$state` is a live wire, every change is delivered to the DOM.



① **Summary.** A plain `let` is a dead wire: change it and

the page never hears about it. `$state` makes the wire live, so every change is delivered to the screen and you never touch the DOM by hand again.

1 — Reactivity: \$state

STATE MANAGEMENT

Q. How do you declare a variable that automatically updates the DOM when its value changes in Svelte 5?

ANSWER. Normally, JavaScript variables are completely dead: changing a `let` variable does not update your UI. You use the `$state` rune to wrap your initial value, turning it into a live variable that Svelte tracks. Any time that value changes, Svelte instantly updates the DOM for you.

```
<script>
  let count = $state(0);
  let user = $state({ name: 'Nikos' });

  function update() {
    count++;
    user.name = 'Hugo'; // Deeply reactive
  }
</script>
```

App.svelte

DIAGRAM CONCEPT. The Live Feed: `$state` is the news wire desk, and every `{count}` in the markup is a subscriber; one update crosses the wire and every subscriber re-renders.

THE LIVE FEED

THE WIRE • ONE SOURCE

```
let count = $state(0)
```



```
<h1>{count}</h1>
```

SUBSCRIBER • RE-RENDERS

```
<button>{count}</button>
```

SUBSCRIBER • RE-RENDERS

ONE UPDATE • EVERY SUBSCRIBER RE-RENDERS

❶ **Summary.** A plain `let` is a dead wire: change it and the page never hears about it. `$state` makes the wire live, so every change is delivered to the screen

and you never touch the DOM by hand again.